Ritmo 2.0
Relatório de evolução e testes

Diego Felipe Anjos de Oliveira
Organizador de estudos | Projeto individual | 30/10/2026

Evolução funcional	Testes SQLite	Interface
3 funcionalidades novas	24 aprovados / 0 falhas	14 verificações concluídas

1. Melhorias implementadas

Funcionalidade	Comportamento implementado
Agenda de revisões	CRUD de revisões por disciplina, data e prioridade. A conclusão confirma o tempo real e cria uma sessão em uma transação, sem duplicar o estudo.
Cronômetro de foco	Duração configurável, pausa, retomada e descarte. Estado persistido em SQLite; o tempo é recuperado após fechar e reabrir, limitado ao alvo. Salvamento com confirmação.
Metas por sessões	Novo alvo por quantidade de sessões, além de tempo. Progresso calculado pelo histórico da disciplina e do período; filtros por tipo e busca por título.

2. Herança e polimorfismo

Os pontos indicados na avaliação anterior foram incorporados ao domínio: **TimeGoal** e **SessionGoal** herdam de **Goal**. As subclasses sobrescrevem **currentValue()**, **targetValue()** e **progressLabel()**. O serviço usa a mesma chamada de método para os dois tipos.

```
const goals = [new TimeGoal(data), new SessionGoal(data)];
const metrics = { studied_minutes: 60, studied_sessions: 1 };
goals.map(goal => goal.progress(metrics)); // [50, 25]
```

O teste usa meta de 120 minutos e meta de quatro sessões. O estado fica em campos privados; toRecord() fornece uma cópia. Evidências: src/models/GoalFactory.js, TimeGoal.js, SessionGoal.js e tests/evolution.test.js.

3. Interface, consultas e persistência

Foram adicionados atalhos para planejamento, alertas de revisões do dia e atrasadas, filtros recolhíveis e limpeza de filtros. Consultas incluem intervalo personalizado, duração, tipo de meta e prioridades. SQL usa parâmetros, filtros e ordenação permitida. A migração do schema 1 para 2 mantém os registros anteriores.

Validação e disponibilização

Resultados observados e etapas necessárias para concluir a entrega.

Área	Verificação executada	Resultado
CRUD e filtros	Disciplinas, sessões, metas e revisões; consultas e ordenação SQL	Aprovado
POO	Subclasses, sobrescrita, cálculo polimórfico e encapsulamento	Aprovado
Integridade	Migração v1-v2; transações; rollback em falha simulada; proteção contra duplicidade	Aprovado
Persistência	Fechamento e reabertura de arquivo SQLite; agenda, foco e metas preservados	Aprovado
Interface	14 fluxos no Expo web, em larguras de 390 e 360 pixels	Sem erros JS não tratados
Android / JavaScript	Exportação Metro/Hermes: npm run export:android	Bundle gerado
APK pelo EAS	Tentativa exigiu autenticação em conta Expo	Pendente de login e build
Instalação no Android	Aparelho ou emulador ainda precisa executar o APK real	Pendente

4. Alcance das evidências

As nove capturas fornecidas são da aplicação real no Expo web. O teste do fim do foco usa relógio acelerado em 65 segundos; isso não representa estudo real. A conferência local das dependências ficou alinhada aos metadados do SDK; a consulta remota sofreu timeout de rede.

O APK ainda não foi gerado. A tentativa do EAS retornou: “An Expo user account is required to proceed.” O perfil preview está configurado com android.buildType = apk e developmentClient = false. O bundle exportado não substitui um APK instalável.

5. Finalização e demonstração individual

1. Executar GERAR_APK.cmd, autenticar a conta Expo e concluir o EAS Build.
2. Guardar o APK e o ID do build; o download registra SHA-256.
3. Instalar no Android e abrir pelo ícone próprio, com Metro e Expo Go encerrados.
4. Demonstrar CRUD, agenda, foco, filtros, persistência e operação em modo avião.
5. Completar evidencias/REGISTRO_DA_INSTALACAO.md e anexar capturas reais.

As atualizações preservam dados quando mantêm o armazenamento, pacote e assinatura. Expo Go e APK têm armazenamentos separados; os dados não são transferidos automaticamente entre eles.

Referências: docs.expo.dev/build-reference/apk/ e docs.expo.dev/build/setup/.
Código e resultados: pastas src/, tests/, capturas/ e evidencias/.